

فاز پنجم

کیوان ثاقب مجیدزاده

موضوع برنامه

هدف این برنامه ساخت خودکار یک proof معتبر برای یک استنتاج معتبر در سامانه Natural Deduction است. ورودی شامل چند مقدمه و یک نتیجه است:

$$\varphi_1, \varphi_2, \dots, \varphi_n \models \psi.$$

خروجی، تعداد گام‌ها و سپس گام‌های proof را چاپ می‌کند. هر گام یا یکی از مقدمات ورودی است، یا با استفاده از یکی از قوانین مجاز و reference به گام‌های قبلی ساخته می‌شود. آخرین گام باید دقیقاً فرمول نتیجه باشد. کد ابتدا فرمول‌ها را به AST تبدیل می‌کند. سپس کلاس ProofSearch با ترکیب Forward Search و اولویت‌دهی بر اساس ساختار goal، فرمول‌های قابل استخراج یا قابل ساخت را تولید می‌کند تا به نتیجه برسد.

قالب ورودی

خط اول شامل عدد n ، یعنی تعداد مقدمات است:

$$1 \leq n \leq 10.$$

در n خط بعدی، مقدمات به ترتیب ورودی قرار دارند و آخرین خط شامل فرمول نتیجه است:

```
n
premise_1
premise_2
...
premise_n
goal
```

طول هر مقدمه و نتیجه حداکثر ۱۰۰ کاراکتر است. صورت سؤال تضمین می‌کند که حداقل یک proof معتبر با قوانین مجاز وجود دارد و تعداد گام‌های آن از ۱۰۰۰ بیشتر نیست.

نمادهای قابل قبول

نماد متنی	معنی
a تا z	متغیر گزاره‌ای تک حرفی
T	ثابت درست
F	ثابت نادرست
$\neg A$	نقیض A
$(A \& B)$	عطف $A \wedge B$
$(A B)$	فصل $A \vee B$
$(A \rightarrow B)$	شرطی $A \rightarrow B$
$(A \leftrightarrow B)$	دوشرطی $A \leftrightarrow B$

زیرفرمول‌های دوتایی باید به طور کامل داخل پرانتز قرار گیرند. نقیض مستقیماً پیش از زیرفرمول بعدی نوشته می‌شود. عبارت‌های اتمی مانند p ، T و F نیز بدون پرانتز هستند.

قالب خروجی

خط اول خروجی تعداد گام‌های proof است. هر گام بدون reference به صورت زیر چاپ می‌شود:

i. formula [rule]

اگر قانون به یک یا چند reference نیاز داشته باشد، قالب زیر استفاده می‌شود:

i. formula [rule: refs]

شماره گام‌ها از یک شروع می‌شود. شماره‌های داخل refs با کاما و بدون فاصله از هم جدا می‌شوند. تمام reference‌ها باید به گام‌های قبلی اشاره کنند.

گام‌های اول تا n دقیقاً مقدمات ورودی، با همان ترتیب و با قانون Premise هستند. آخرین گام نیز باید دقیقاً فرمول نتیجه باشد.

چندجوابی بودن خروجی

برای یک ورودی ممکن است چند proof معتبر متفاوت وجود داشته باشد. هر خروجی که قوانین مجاز، ترتیب reference‌ها، محدودیت ۱۰۰۰ گام و قرارگرفتن نتیجه در گام آخر را رعایت کند پذیرفته است. برنامه لازم نیست کوتاه‌ترین proof ممکن را تولید کند.

قوانین مجاز

کد قانون	نحوه استفاده
Premise	فرمول یکی از مقدمات ورودی است.
&I	از φ و ψ ، فرمول $\varphi \wedge \psi$ ساخته می‌شود.
&E	از $\varphi \wedge \psi$ می‌توان φ یا ψ را استخراج کرد.
I	از φ می‌توان $\varphi \vee \psi$ یا $\psi \vee \varphi$ را ساخت.
->E	با Modus Ponens، از $\varphi \rightarrow \psi$ و φ ، نتیجه ψ به دست می‌آید.
!E	از φ و $\neg\varphi$ ، ثابت F به دست می‌آید.
FE	از ثابت F می‌توان هر فرمول دلخواهی را نتیجه گرفت.
<->E	از $\varphi \leftrightarrow \psi$ می‌توان $\varphi \rightarrow \psi$ یا $\psi \rightarrow \varphi$ را ساخت.
!!	نقیض مضاعف را حذف می‌کند یا یک نقیض مضاعف می‌افزاید.

قوانینی که به فرض موقت و زیراثبات نیاز دارند، در این برنامه استفاده نمی‌شوند. در نتیجه، قوانینی مانند - Introduction, >Introduction, |Elimination و Introduction! در جست‌وجو وجود ندارند.

محدودیت‌های تضمین‌شده ورودی

صورت سؤال برای امکان ساخت proof بدون زیراثبات، دو محدودیت مهم اعمال می‌کند:

۱. هیچ مقدمه‌ای در هیچ سطحی شامل عملگر فصل نیست. عملگر فصل می‌تواند در نتیجه وجود داشته باشد و با |I ساخته شود.

۲. هر زیرفرمول نتیجه که عملگر اصلی آن نقیض، شرطی یا دوشروطی است، باید با برابری ساختاری در حداقل یکی از مقدمات نیز ظاهر شده باشد.

به همین دلیل، برنامه لازم نیست این ساختارها را از صفر معرفی کند و می‌تواند آن‌ها را از مقدمات استخراج یا استفاده کند.

ساختار کلی اجرای برنامه

مراحل اجرای برنامه به ترتیب زیر است:

- خواندن مقدمات و نتیجه در تابع main؛
- تجزیه فرمول‌ها و ساخت AST؛
- ثبت مقدمات به عنوان گام‌های ابتدایی؛
- جمع‌آوری زیرفرمول‌های مرتبط؛
- مرتب کردن نامزدهای قوانین معرفی با یک Heuristic؛

۰۶. اجرای قوانین حذف روی فرمول‌های شناخته‌شده؛

۰۷. تشخیص تناقض و تولید F در صورت امکان؛

۰۸. اجرای Modus Ponens؛

۰۹. اجرای قوانین معرفی روی نامزدهای مرتبط؛

۰۱۰. تکرار مراحل تا تولید goal؛

۰۱۱. قراردادن نتیجه در آخرین گام و قالب‌بندی خروجی.

نمایش فرمول با AST

کلاس داده‌ای Formula هر گره از AST را نمایش می‌دهد:

ویژگی	توضیح
kind	نوع گره، مانند var، const، not، and، or، imp یا iff
value	نام متغیر یا مقدار ثابت
left	فرزند چپ یا تنها فرزند گره نقیض
right	فرزند راست در عملگرهای دوتایی

گزینه frozen=True باعث می‌شود فرمول‌ها تغییرناپذیر و قابل ذخیره در set یا استفاده به عنوان کلید dict باشند. این ویژگی برای جلوگیری از تولید دوباره فرمول‌های یکسان ضروری است.

توابع make_variable، make_constant، make_not و make_binary سازنده‌های کمکی گره‌ها هستند.

کلاس FormulaParser

این کلاس فرمول را از چپ به راست و به صورت بازگشتی تجزیه می‌کند. position محل فعلی خواندن متن را نگه می‌دارد. متد parse_formula یکی از متغیر، ثابت، نقیض یا فرمول دوتایی را تشخیص می‌دهد.

در فرمول دوتایی، ابتدا طرف چپ، سپس عملگر و بعد طرف راست خوانده می‌شود. عملگرهای بلندتر باید زودتر بررسی شوند؛ بنابراین ترتیب تشخیص چنین است:

```
<->
->
&
|
```

متد parse پس از ساخت فرمول بررسی می‌کند که همه کاراکترهای ورودی مصرف شده باشند.

تابع `render_formula`

این تابع AST را دوباره به نمادگذاری متنی صورت سؤال تبدیل می کند. متغیرها و ثابت ها مستقیماً چاپ می شوند. برای نقیض، علامت ! پیش از فرزند قرار می گیرد و عملگرهای دوتایی با یک جفت پرانتز چاپ می شوند. نگاشت نوع داخلی عملگرها به نماد خروجی چنین است:

نوع داخلی	نماد خروجی
and	&
or	
imp	->
iff	<->

کلاس `ProofStep`

هر گام `proof` با این کلاس ذخیره می شود. ویژگی های آن عبارت اند از:

- `formula`: فرمول تولیدشده در این گام؛
 - `rule`: کد قانون استفاده شده؛
 - `references`: شماره گام های قبل مورد استفاده.
- برای قانون `Premise`، فهرست `references` خالی است.

آماده سازی کلاس ProofSearch

سازنده ProofSearch مقدمات و goal را دریافت می کند. ابتدا تمام مقدمات با قانون Premise به فهرست steps افزوده می شوند. نگاشت known نیز هر فرمول شناخته شده را به شماره اولین گامی که آن را تولید کرده است مرتبط می کند.

اگر یک مقدمه چند بار تکرار شده باشد، همه رخدادهای آن در خروجی مقدمات حفظ می شوند، اما known فقط شماره اولین رخداد را نگه می دارد.

فرمول های مرتبط

مجموعه relevant شامل تمام زیرفرمول های مقدمات و نتیجه است. محدود کردن قوانین معرفی به این مجموعه از ساخت فرمول های نامرتب و ایجاد فضای جست و جوی نامحدود جلوگیری می کند.

اگر یک فرمول دوشروطی در این مجموعه وجود داشته باشد، دو فرمول شرطی حاصل از قانون $\rightarrow E$ نیز به مجموعه افزوده می شوند:

$$\varphi \leftrightarrow \psi \implies \varphi \rightarrow \psi, \quad \psi \rightarrow \varphi.$$

مجموعه goal_subformulas فقط زیرفرمول های نتیجه را نگه می دارد و برای هدایت جست و جو استفاده می شود.

نامزدهای قوانین معرفی

فهرست introduction_candidates شامل فرمول های مرتبطی است که با یکی از قوانین $\&I$ ، I یا $!!$ قابل ساخت هستند. این فهرست با تابع candidate_priority مرتب می شود.

اولویت هر نامزد یک سه تایی است:

$$(\text{goal_priority}, \text{formula_size}, \text{rendered_text}).$$

ابتدا زیرفرمول های goal، سپس فرمول های کوچک تر و در پایان ترتیب متن در نظر گرفته می شود. این روش Heuristic اصلی برنامه است و باعث می شود جست و جو زودتر به ساختار نتیجه نزدیک شود.

جلوگیری از تکرار و حلقه

متد add_step پیش از افزودن هر فرمول بررسی می کند که آیا آن فرمول در known وجود دارد یا خیر. اگر فرمول قبلاً تولید شده باشد، گام جدید ساخته نمی شود.

این روش از تکرار حالت ها جلوگیری می کند و به ویژه مانع ایجاد حلقه بی نهایت با قانون $!!$ می شود؛ زیرا بدون این کنترل می توان زنجیره ای مانند زیر ساخت:

$$p, \quad \neg p, \quad \neg\neg\neg p, \quad \dots$$

تابع `collect_subformulas`

این تابع فرمول و تمام فرزندان آن را به صورت بازگشتی پیمایش می‌کند. هر فرمول فقط یک بار در مجموعه مقصد قرار می‌گیرد. این تابع برای ساخت `relevant` و `goal_subformulas` استفاده می‌شود.

تابع `formula_size`

اندازه یک فرمول برابر تعداد گره‌های AST آن در نظر گرفته شده است. تابع از مقدار یک برای گره فعلی شروع می‌کند و اندازه فرزندان چپ و راست را به صورت بازگشتی اضافه می‌کند. این اندازه یکی از اجزای Heuristic مرتب‌سازی نامزدها است.

حلقه اصلی generate

متد generate تا زمانی که goal شناخته نشده باشد، قوانین مجاز را در چند مرحله اجرا می کند. متغیر changed مشخص می کند که آیا در دور فعلی حداقل یک فرمول جدید تولید شده است یا خیر. اگر هیچ فرمول جدیدی تولید نشود و goal نیز شناخته نشده باشد، برنامه خطای RuntimeError ایجاد می کند. طبق تضمین صورت سؤال، این حالت برای ورودی معتبر رخ نمی دهد.

استفاده مستقیم از FE

اگر ثابت F از قبل شناخته شده باشد، برنامه مستقیماً goal را با قانون FE تولید می کند:

```
goal [FE: false_step]
```

این بررسی در ابتدای هر دور انجام می شود تا پس از رسیدن به تناقض، نیازی به ادامه جست و جو نباشد.

قوانین حذف روبه جلو

برنامه ابتدا یک تصویر ثابت از known می سازد تا هنگام افزودن گام های جدید، پیمایش نگاشت تغییر نکند. سپس قوانین زیر اعمال می شوند:

- برای هر عطف شناخته شده، هر دو طرف با E & استخراج می شوند.
- برای هر دوشروطی شناخته شده، دو جهت شرطی با E <-> ساخته می شوند.
- از هر نقیض مضاعف شناخته شده، فرمول داخلی با !! استخراج می شود.

اگر در میان این مراحل goal تولید شود، دور جست و جو متوقف می شود.

تشخیص تناقض با E!

برنامه تمام فرمول های شناخته شده را بررسی می کند. برای هر φ ، فرمول $\neg\varphi$ ساخته و در known جست و جو می شود. اگر هر دو وجود داشته باشند، ثابت F با قانون E! تولید می شود:

$$\varphi, \neg\varphi \vdash F.$$

پس از تولید F، برنامه بلافاصله goal را با FE می سازد.

اجرای Modus Ponens

در مرحله بعد، تمام فرمول‌های شرطی شناخته شده بررسی می‌شوند. اگر $\psi \rightarrow \varphi$ و φ هر دو در known باشند، ψ با قانون $\rightarrow E$ تولید می‌شود.

reference اول، شماره گام فرمول شرطی و reference دوم، شماره گام بخش مقدم آن است. این ترتیب دقیقاً با قرارداد خروجی مطابقت دارد.

قوانین معرفی

در مرحله معرفی، فقط اعضای introduction_candidates بررسی می‌شوند:

- یک عطف با $\&I$ زمانی ساخته می‌شود که هر دو طرف آن شناخته شده باشند.
- یک فصل با $|I$ زمانی ساخته می‌شود که حداقل یکی از دو طرف آن شناخته شده باشد.
- یک نقیض مضاعف با $!!$ زمانی ساخته می‌شود که فرمول داخلی شناخته شده باشد.

ترتیب نامزدها باعث می‌شود ساخت زیرفرمول‌های goal پیش از فرمول‌های مرتبط دیگر امتحان شود. این بخش نقش Backward Search را به صورت هدایت نامزدها ایفا می‌کند، در حالی که قواعد حذف و Modus Ponens بخش Forward Search هستند.

محدودیت تعداد گام‌ها

پس از هر دور، تعداد گام‌ها بررسی می‌شود. اگر تعداد گام‌ها از ۱۰۰۰ بیشتر شود، خطای RuntimeError ایجاد می‌شود. پس از پایان جست‌وجو نیز این بررسی یک بار دیگر انجام می‌شود.

متد ensure_goal_is_last

ممکن است goal پیش‌تر تولید شده باشد، اما آخرین گام فهرست نباشد. صورت سؤال الزام می‌کند که آخرین گام دقیقاً نتیجه باشد. این متد تلاش می‌کند نتیجه را با یک قانون معتبر دوباره در انتهای proof تولید کند. ترتیب روش‌های بازتولید چنین است:

۱. استفاده از FE در صورت وجود F؛
۲. استخراج از عطف با $\&E$ ؛
۳. استخراج یک جهت دوشروطی با $\rightarrow E$ ؛
۴. حذف نقیض مضاعف با $!!$ ؛
۵. ساخت عطف با $\&I$ ؛
۶. ساخت فصل با $|I$ ؛

۰۷. افزودن نقیض مضاعف با !!؛

۰۸. استفاده از Modus Ponens.

اگر هیچ کدام از این روش‌های یک گامی قابل استفاده نباشند، متد ابتدا از goal، نقیض مضاعف آن را می‌سازد و سپس دوباره با حذف نقیض مضاعف به goal می‌رسد. این مسیر دوگامی تضمین می‌کند آخرین گام فرمول نتیجه باشد.

تابع `format_proof`

این تابع ابتدا تعداد گام‌ها را در خط اول قرار می‌دهد. سپس هر گام را شماره‌گذاری و فرمول آن را با `render_formula` به متن تبدیل می‌کند.

اگر گام `reference` داشته باشد، شماره‌ها با کاما و بدون فاصله متصل می‌شوند:

3. q [->E: 1,2]

اگر `reference` وجود نداشته باشد، فقط نام قانون چاپ می‌شود:

1. p [Premise]

توابع `main` و `solve`

تابع `solve` متن مقدمات و نتیجه را با `FormulaParser` به AST تبدیل می‌کند. سپس یک نمونه از `ProofSearch` می‌سازد، `generate` را اجرا می‌کند و نتیجه را با `format_proof` قالب‌بندی می‌کند.

تابع `main` ابتدا تعداد مقدمات، سپس مقدمات و در پایان نتیجه را با `input` می‌خواند و خروجی `solve` را چاپ می‌کند.

اجرای نمونه اول: Modus Ponens

ورودی:

2
($p \rightarrow q$)
p
q

دو گام نخست مقدمات هستند. از فرمول شرطی گام اول و بخش مقدم آن در گام دوم، نتیجه با $\rightarrow E$ ساخته می شود:

3
1. ($p \rightarrow q$) [Premise]
2. p [Premise]
3. q [$\rightarrow E$: 1,2]

اجرای نمونه دوم: حذف عطف

ورودی:

1
($p \& q$)
p

از عطف موجود در گام اول، طرف چپ با $\& E$ استخراج می شود:

2
1. ($p \& q$) [Premise]
2. p [$\& E$: 1]

اجرای نمونه سوم: رسیدن به F

ورودی:

2
p
!p
q

با داشتن p و $\neg p$ ، قانون $\neg E$ ثابت F را تولید می کند. سپس با FE می توان نتیجه دلخواه q را ساخت:

4
1. p [Premise]
2. !p [Premise]
3. F [$\neg E$: 1,2]
4. q [FE: 3]

پیچیدگی و فضای جست‌وجو

اگر تعداد فرمول‌های موجود در مجموعه relevant را R و تعداد گام‌های تولیدشده را S در نظر بگیریم، هر دور جست‌وجو چند پیمایش روی known و introduction_candidates انجام می‌دهد.

بررسی قوانین حذف، تناقض و شرطی در هر دور تقریباً متناسب با تعداد فرمول‌های شناخته‌شده است. بررسی قوانین معرفی نیز حداکثر R نامزد را در نظر می‌گیرد. تعداد گام‌ها طبق محدودیت مسئله حداکثر ۱۰۰۰ است.

استفاده از known باعث می‌شود هر فرمول حداکثر یک بار به عنوان یک گام جدید وارد جست‌وجو شود. مجموعه relevant نیز فضای ساخت فرمول‌های معرفی‌شده را محدود می‌کند. بدون این دو محدودیت، قانون !! و قوانین معرفی می‌توانستند فضای جست‌وجو را نامحدود کنند.

محدودیت‌های برنامه

- برنامه فقط قوانین مجاز مشخص‌شده در صورت سؤال را پیاده‌سازی می‌کند.
- زیراثبات و فرض موقت پشتیبانی نمی‌شوند.
- مقدمات نباید شامل عملگر فصل باشند.
- ساختارهای نقیض، شرطی و دوشروطی مورد نیاز نتیجه باید در مقدمات قابل دسترسی باشند.
- صورت سؤال وجود یک proof معتبر را تضمین می‌کند.
- این پیاده‌سازی Sanity Check معنایی جداگانه انجام نمی‌دهد و به تضمین معتبر بودن استنتاج ورودی متکی است.
- خروجی حداکثر می‌تواند ۱۰۰۰ گام داشته باشد.
- نام هر متغیر فقط یک حرف کوچک انگلیسی از a تا z است.
- فرمول‌های دوتایی باید به‌طور کامل داخل پرانتز قرار گیرند.
- افزایش حد بازگشت به 10000 امکان پردازش فرمول‌های عمیق‌تر را فراهم می‌کند، اما ورودی بسیار عمیق همچنان می‌تواند منابع زیادی مصرف کند.